



Homework

Async and Blocking Task Scheduling

Recap

1. Java Executor Framework
2. Task Scheduling
3. Async Task Scheduling
4. Tomcat Server Threading Model

Homework

1. Write a program that uses a `ThreadPoolExecutor` with the following configuration:
 - Core pool size: 2
 - Maximum pool size: 5
 - Keep-alive time: 30 seconds
 - A bounded queue with a size of 10 Simulate 20 tasks, each taking 2 seconds to complete. Observe how threads are utilized and what happens to the excess tasks.

2. Write a Spring Boot application with three `@Async` methods:
 - `fetchUserData()`: Simulate fetching user details (2 seconds delay).
 - `fetchOrderData()`: Simulate fetching order details (3 seconds delay).
 - `fetchPaymentData()`: Simulate fetching payment history (4 seconds delay). Use these methods in a `getUserReport()` method to fetch all data concurrently and return the combined result.

Homework

3. Implement a custom rejection policy in a `ThreadPoolExecutor` that logs a warning message and retries rejected tasks after 1 second.
4. Implement a dynamic task scheduler using Spring's `TaskScheduler` that allows the user to call an API that adds a task to send an email after 10 seconds.

